ProbOS — Month 2, Week 7
FastAPI Service Layer

Nisong Monyimba

July 3, 2026

Week 7 goal

Expose the kernel over HTTP so it is usable outside a Python REPL – the first step toward external/enterprise-facing usage referenced in `docs/monthly_plans/overall/main.tex`.

Day-by-day plan

Day 1 – FastAPI scaffold

- Set up `python/server/` package structure
- Basic FastAPI app, health check endpoint
- Pydantic schemas mirroring `Model`/`Distribution` constructor signatures

Day 2 – /simulate endpoint

- `POST /simulate`: accept model name, priors, N, `n_steps`; run `MonteCarloEngine`; return percentiles + convergence summary as JSON
- Input validation: sane upper bounds on N/`n_steps` to prevent resource exhaustion (per `docs/standards/qualit` Section 8, deferred item now becoming relevant)

Day 3 – /sensitivity endpoint

- `POST /sensitivity`: run `SobolSensitivity`, return S_1/S_T indices as JSON

Day 4 – /filter endpoint

- `POST /filter`: run `ParticleFilter` against posted observation data, return posterior mean/std trajectory

Day 5 – Integration tests

- `python/tests/test_server.py`: FastAPI `TestClient`-based tests for all endpoints
- Confirm JSON responses match direct Python kernel calls numerically

Day 6 – Pre-week audit + CI integration

- Run `pre_week_audit.sh` before considering the week complete
- Extend CI to install and test the server layer
- Extend `docs/standards/quality_standards.md` Section 8 with the now-relevant API/service checks (previously listed as deferred)

Day 7 – Study guide + documentation

- `docs/study/study_guide.md` entries: FastAPI documentation, Pydantic validation patterns
- Week 7 retrospective appended to this document

Exit criteria

`curl -X POST localhost:8000/simulate ...` returns valid JSON matching a direct Python call to the same `MonteCarloEngine`, for all three endpoints (`/simulate`, `/sensitivity`, `/filter`).

What was actually accomplished (retrospective)

- `python/server/schemas.py`: Pydantic request/response schemas for all four endpoints, with resource-exhaustion bounds enforced via `Field(ge=..., le=...)` at the HTTP boundary – exactly the previously-deferred check from `quality_standards.md` Section 8, now active
- `python/server/main.py`: `GET /health`, `POST /simulate`, `POST /sensitivity`, `POST /filter` – each a thin translation layer over an existing kernel object, no new kernel logic
- Kernel-raised `ValueError` surfaces as HTTP 422 (not an unhandled 500); unknown `model_name` surfaces as HTTP 404
- 19 integration tests, including three that directly compare HTTP JSON responses against a matching direct Python kernel call at `rtol=1e-10` – proving zero numerical discrepancy introduced by the service layer, not just "returns 200"
- Additionally smoke-tested with a real `uvicorn` process over an actual HTTP socket (`/health` and `/simulate`), not just FastAPI's in-process `TestClient`
- `quality_standards.md` Section 8 updated: the previously-deferred API/service checks are now active and checked off, each pointing to the specific test that verifies it
- CI extended across all three relevant jobs (`python-tests`, `integration`, `quality-audit`) to install and test the new server layer

Numbers at Week 7 close

- Python tests: 341/341 passing (322 + 19 new)
- C++ tests: 13/13 passing
- mypy strict: 0 errors (full `python/` tree including `python/server/`)

- ruff: 0 warnings
- `pre_week_audit.sh`: 21/21 checks passed

Carried into later weeks

Only the `battery` model is currently registered in the server layer's model registry. Extending it to the Week 4 cross-discipline models (option pricer, ED queue, clinical trial) is a natural, low-risk follow-up once genuinely needed. The service layer itself is complete and satisfies its exit criteria exactly as planned.